INTELLIGENT HYBRID FRAMEWORK FOR REAL-TIME DETECTION AND MITIGATION OF SQL INJECTION IN WEB APPLICATIONS

ML – MODEL

How it works step-by-step

SYSTEM COMPONENTS

Flask Proxy Gateway

- Acts as a firewall.
- Intercepts all incoming user inputs before reaching the backend.

Hybrid Detection System

- Rule-based Filter: Uses regex and keyword matching to detect known SQLi patterns.
- Machine Learning Classifier: Uses TF-IDF + Random Forest to detect complex or new attacks.
- Mitigation Engine: If a query is harmful → blocks it or sanitizes it before sending to the database.

Secure Backend

- Uses prepared statements and parameterized queries to stop injection even if something slips through.

Architecture / Workflow

1. User enters data →
2. Flask proxy intercepts →
3. Rule-based filter checks known patterns →
4. If safe → goes to ML classifier →
5. ML model assigns a “risk score” →
6. Mitigation engine blocks or cleans risky queries →
7. Secure backend executes only safe queries →
8. Response sent back to user.

Algorithm

1.Rule-Based Filtering
2.Random Forest Classifier
3.Mitigation Logic - Blocks or sanitizes malicious queries
Real-time protection

TF-IDF → TERM FREQUENCY – INVERSE DOCUMENT FREQUENCY

It’s a numerical statistic used in Natural Language Processing (NLP) and Machine Learning to convert text into meaningful numerical values (features).

In our project, it helps convert SQL queries (text) into numerical vectors for the ML model.

TF – Term Frequency

MEASURES HOW FREQUENTLY A TERM (WORD) APPEARS IN A DOCUMENT

$$\mathrm{TF}(t) = \frac{\text{Number of times term t appears in query}}{\text{Total number of terms in the query}}$$

IDF – INVERSE DOCUMENT FREQUENCY

MEASURES HOW RARE OR UNIQUE A TERM IS ACROSS ALL DOCUMENTS

$$\mathrm{IDF}(t) = \log\left(\frac{\text{Total number of queries}}{\text{Number of queries containing term t}}\right)$$

COMBINED TF-IDF FORMULA

$$\text{TF-IDF}(t) = \text{TF}(t) \times \text{IDF}(t)$$

EACH SQL QUERY IS CONVERTED INTO A TF-IDF VECTOR – A LIST OF WEIGHTED NUMBERS.
THE LOGISTIC REGRESSION & RANDOM FOREST CLASSIFIER USES THIS VECTOR TO UNDERSTAND WHICH PATTERNS ARE TYPICAL OR SUSPICIOUS.

HOW FINAL_SCORE IS CALCULATED

THE FORMULA COMBINES MACHINE LEARNING PREDICTION AND ANOMALY DETECTION SCORE, AND GIVES EXTRA WEIGHT TO RULE-BASED MATCHES IF DETECTED.

$$\text{final\_score} = (w_1 \times \text{ml\_prob}) + (w_2 \times \text{anomaly\_score}) + (w_3 \times \text{rule\_flag})$$

ml_prob → probability of SQLi from the ML model

anomaly_score → statistical outlier score (unusual pattern)

rule_flag → 1 if rule-based regex matched, else 0

w_1, w_2, w_3 → weights, e.g. 0.4, 0.3, 0.3

Threshold: if `final_score ≥ 0.5` → **block**, else **allow**

STEP 1. DATASET EXAMPLE

LET’S SAY WE HAVE 3 SQL QUERIES (DOCUMENTS)

Document	SQL Query
D1	`SELECT * FROM users WHERE id=1`
D2	`SELECT * FROM users WHERE id='1' OR '1'='1'`
D3	`DROP TABLE users`

STEP 2. EXTRACT TERMS

Let’s consider only a few key terms (words):

`SELECT`, `FROM`, `users`, `WHERE`, `OR`, `DROP`, `TABLE`.

STEP 3. CALCULATE TF (TERM FREQUENCY)

TF = (NUMBER OF TIMES A TERM APPEARS IN THE DOCUMENT) / (TOTAL NUMBER OF TERMS IN THAT DOCUMENT)

LET'S CALCULATE FOR DOCUMENT D2 = "SELECT * FROM USERS WHERE ID='1' OR '1'='1'"

D3 = DROP TABLE

Term	Count	Total Terms = 7	TF
SELECT	1	7	1/7 = **0.1429**
FROM	1	7	**0.1429**
users	1	7	**0.1429**
WHERE	1	7	**0.1429**
OR	1	7	**0.1429**
DROP	0	7	**0.0000**
TABLE	0	7	**0.0000**

STEP 4. CALCULATE IDF (INVERSE DOCUMENT FREQUENCY)

IDF MEASURES HOW RARE OR UNIQUE A TERM IS ACROSS ALL DOCUMENTS.

$$\mathrm{IDF}(t) = \log_{10}\left(\frac{N}{n_t}\right)$$

N = TOTAL NUMBER OF DOCUMENTS (HERE = 3)
N(T) = NUMBER OF DOCUMENTS CONTAINING THE TERM T

Term	Appears in how many docs (n_t)	IDF = $\log_{10}(3 / n_t)$
SELECT	2	$\log_{10}(3/2)$ = **0.1761**
FROM	2	**0.1761**
users	3	$\log_{10}(3/3)$ = **0.0000**
WHERE	2	**0.1761**
OR	1	$\log_{10}(3/1)$ = **0.4771**
DROP	1	**0.4771**
TABLE	1	**0.4771**

STEP 5. CALCULATE TF-IDF FOR D2

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

Term	TF	IDF	TF × IDF = TF-IDF
SELECT	0.1429	0.1761	**0.0251**
FROM	0.1429	0.1761	**0.0251**
users	0.1429	0.0000	**0.0000**
WHERE	0.1429	0.1761	**0.0251**
OR	0.1429	0.4771	**0.0682**
DROP	0.0000	0.4771	**0.0000**
TABLE	0.0000	0.4771	**0.0000**

STEP 6. INTERPRETATION

THE HIGHEST TF-IDF SCORE IS FOR THE WORD “OR” = 0.0682

This term is rare overall but present in a known SQL injection pattern (`OR '1'='1'`).

Hence, it becomes a **strong indicator of SQL injection** for the ML model.

Thus, TF-IDF helps the **Logistic Regression model** learn:

- Which terms are **normal SQL syntax** (low weights)
- Which terms are **anomalous or dangerous** (high weights)

STEP 7. LOGISTIC REGRESSION DECISION

The TF-IDF vector `[0.0251, 0.0251, 0, 0.0251, 0.0682, 0, 0]` is fed into the Logistic Regression model.

The model calculates:

$$\text{Prediction} = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

If the probability > 0.5 → **SQL Injection Detected**

For D2, since “OR” has a high TF-IDF and strong positive coefficient weight,

→ The model predicts `SQLi = 1`.

WHAT LOGISTIC REGRESSION DOES

Logistic Regression doesn't just classify 0 or 1 directly.

It calculates a **probability score** between 0 and 1 that tells **how likely** a query is an SQL injection.

It uses a **mathematical function** called the **sigmoid function**:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

Where:

- **x** → TF-IDF vector (numerical representation of your query)
- **w** → weights learned by the model for each term (feature importance)
- **b** → bias/intercept
- **P(y=1|x)** → probability that the query is SQL Injection

REAL EXAMPLE USING QUERY

Let's take your malicious query:
`' OR '1'='1`

and a normal one:
`alice`

We'll see how the model decides.

STEP 1: TF-IDF REPRESENTATION

Each query is converted into numeric features using **TF-IDF**, which gives higher importance to rare or risky words.

Term	`' OR '1'='1`	`alice`
SELECT	0.00	0.00
FROM	0.00	0.00
users	0.00	0.00
WHERE	0.00	0.00
OR	**0.0682**	**0.00**
1	**0.0450**	**0.00**
=	**0.0450**	**0.00**
Others	0	0

The injection query `' OR '1'='1` has risky terms **"OR"** and **"="** → hence **non-zero TF-IDF values.** "alice" has none → all zeros.

STEP 2: APPLY LOGISTIC REGRESSION EQUATION

Let's assume your model learned weights like this (from training):

Term	Weight (w_i)	Meaning
OR	+2.0	Strong indicator of SQL injection
=	+1.5	Common in injection patterns
1	+0.8	Often used in `1=1` attacks
SELECT / FROM	0.3	Normal query parts
Other terms	0	Neutral
Bias (b)	-0.2	Adjusts threshold

Now compute the **weighted sum**:

$$z = (2.0 \times 0.0682) + (1.5 \times 0.0450) + (0.8 \times 0.0450) - 0.2 = 0.3764$$

STEP 3: APPLY SIGMOID FUNCTION

$$P(y=1|x) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-0.3764}} = 0.5931$$

That means:

59.31% probability that this query is an **SQL Injection.**

STEP 4: DECISION

Rule	Result
If P ≥ 0.5	Block
If P < 0.5	Allow

STEP 5: COMPARE WITH A NORMAL INPUT

Let's test the same model with the query `"alice"` :

All TF-IDF values = 0 →

$$z = (2.0 \times 0) + (1.5 \times 0) + (0.8 \times 0) - 0.2 = -0.2$$

$$P(y=1|x) = \frac{1}{1+e^{-(-0.2)}} = 0.4502$$

Since 0.45 < 0.5 →

✅ **Decision = Allow (Normal Input)**

STEP 6: WHAT HAPPENS IN OUR FRAMEWORK

Stage	Component	What it does
Input Layer	Flask Proxy	Captures incoming query
Feature Extraction	TF-IDF	Converts query text into numeric vector
ML Detection	Logistic Regression	Calculates probability using learned weights
Threshold Decision	Logic	If prob > 0.5 → block
Mitigation	Flask Engine	Sanitizes or blocks query before hitting database

RANDOM FOREST CLASSIFIER – CONCEPT

Random Forest is an **ensemble algorithm** — it combines the results of **many decision trees** to make a final decision.

Each tree votes whether the query is **malicious (1)** or **normal (0)**.

The final probability is the **average of all votes**.

$$P(y = 1|x) = \frac{\text{Number of trees predicting SQLi}}{\text{Total trees}}$$

WHY RANDOM FOREST IS USED IN OUR PROJECT

It **handles both linear and non-linear patterns** — great for text like SQL queries.

It **reduces overfitting** compared to single decision trees.

It can combine **word frequency (TF-IDF)**, **keyword count**, and **rule-based flags** all together.

Works well with small or medium datasets and gives **very high accuracy** (your project achieved 1.0).

HOW IT WORKS IN OUR HYBRID FRAMEWORK

1. The SQL query is first converted into a **TF-IDF vector** (same as for Logistic Regression).
2. Each tree in the forest checks certain **conditions** (like if specific risky words exist).
3. Each tree gives a vote:
    - 1 → SQL Injection (malicious)
    - 0 → Normal (safe)
4. The forest aggregates all votes and outputs a **final probability**.

LET'S TAKE ONE EXAMPLE

We'll use two inputs:

- Query 1 → `' OR '1'='1` (known SQL injection)
- Query 2 → `alice` (normal input)

And assume we have **5 decision trees** in our forest.

STEP 1. TF-IDF INPUT VECTOR

Term	`OR '1'='1`	`alice`
OR	0.0682	0
1	0.045	0
=	0.045	0
users	0	0
DROP	0	0

STEP 2. EACH DECISION TREE VOTES

Tree	Condition Example	`' OR '1'='1` → Output	`alice` → Output
Tree 1	If “OR” & “=” found → SQLi	1	0
Tree 2	If “1=1” pattern exists → SQLi	1	0
Tree 3	If query contains rare term → SQLi	1	0
Tree 4	If length < 2 words → Normal	1	0
Tree 5	If no keyword → Normal	1	0

✅ `' OR '1'='1` = 5 trees vote **SQLi (1)**

✅ `alice` = 0 trees vote SQLi (all **0**)

STEP 3. CALCULATE PROBABILITY

$$P(\text{SQLi}) = \frac{\text{Number of 1 votes}}{\text{Total Trees}}$$

Query	Trees Voting SQLi	Probability (avg)	Decision
`' OR '1'='1`	5 / 5	1.0 (100%)	Block
`alice`	0 / 5	0.0 (0%)	Allow

This matches your Excel log where:

- **ml_prob = 1** for `' OR '1'='1`
- **ml_prob = 0** for `alice`

BEHIND THE SCENES – RANDOM FOREST DECISION LOGIC

Each tree checks rules like:

python

```python
if "OR" in query and "=" in query:
    return 1  # SQLi
elif "UNION" in query or "DROP" in query:
    return 1
else:
    return 0
```

When hundreds of such rules (trees) work together, they capture:

- Known attack patterns (rule-like behavior)
- Subtle word combinations (machine learning behavior)
- Context of input structure (e.g., “WHERE id=1 OR 1=1”)